



```
In [ ]: from sklearn.model_selection import train_test_split
        from sklearn.ensemble import StackingClassifier
        from sklearn.tree import DecisionTreeClassifier
        from sklearn.linear_model import LogisticRegression
        from sklearn.svm import SVC
        from sklearn.metrics import accuracy_score, classification_report
        import seaborn as sns
        from sklearn.preprocessing import LabelEncoder
```

```
In [ ]: df = sns.load_dataset('iris')
```

```
In [ ]: df.head()
```

```
Out[ ]:   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2    setosa
1           4.9           3.0           1.4           0.2    setosa
2           4.7           3.2           1.3           0.2    setosa
3           4.6           3.1           1.5           0.2    setosa
4           5.0           3.6           1.4           0.2    setosa
```

```
In [ ]: X = df.drop('species', axis=1)
        y = df['species']
```

```
In [ ]: le = LabelEncoder()
        y_encoded = le.fit_transform(y)
```

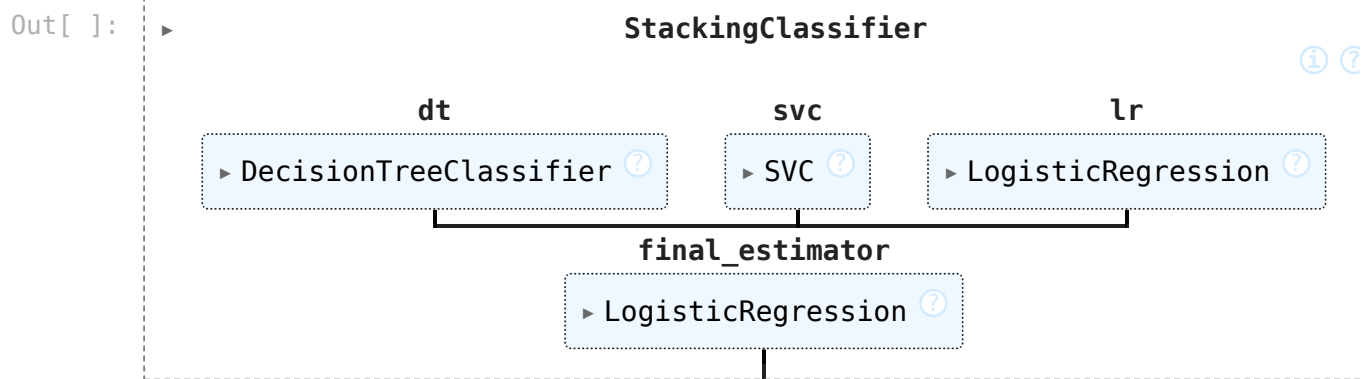
```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2)
```

```
In [ ]: base_learners = [
        ('dt', DecisionTreeClassifier(random_state=42)),
        ('svc', SVC(probability=True, kernel='rbf', random_state=42)),
        ('lr', LogisticRegression(max_iter=1000))
        ]
```

```
In [ ]: meta_learner = LogisticRegression(max_iter=1000)
```

```
In [ ]: stacking_clf = StackingClassifier(
        estimators=base_learners,
        final_estimator=meta_learner,
        cv=5
        )
```

```
In [ ]: stacking_clf.fit(X_train, y_train)
```



```
In [ ]: y_pred = stacking_clf.predict(X_test)
```

```
In [ ]: accuracy = accuracy_score(y_test, y_pred)
```

```
In [ ]: accuracy
```

Out []: 0.9666666666666667

```
In [ ]: from sklearn.ensemble import RandomForestClassifier
```

```
In [ ]: rf_model = RandomForestClassifier(
    n_estimators=100,      # number of trees
    max_depth=None,       # let trees grow fully
    random_state=42,
)
```

```
In [ ]: rf_model.fit(X_train,y_train)
```

Out []:

RandomForestClassifier

RandomForestClassifier(random_state=42)

```
In [ ]: y_pred = rf_model.predict(X_test)
```

```
In [ ]: accuracy = accuracy_score(y_test, y_pred)
```

```
In [ ]: accuracy
```

Out []: 0.9

```
In [ ]: from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
```

```
In [ ]: from xgboost import XGBClassifier
```

```
In [ ]: ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)
```

```
In [ ]: ada_model.fit(X_train,y_train)
```

```
Out[ ]: ▾ AdaBoostClassifier ⓘ ?  
AdaBoostClassifier(n_estimators=100, random_state=42)
```

```
In [ ]: y_pred = ada_model.predict(X_test)
```

```
In [ ]: accuracy = accuracy_score(y_test, y_pred)
```

```
In [ ]: accuracy
```

```
Out[ ]: 0.9333333333333333
```

```
In [ ]: gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, ran
```

```
In [ ]: gb_model.fit(X_train,y_train)
```

```
Out[ ]: ▾ GradientBoostingClassifier ⓘ ?  
GradientBoostingClassifier(random_state=42)
```

```
In [ ]: y_pred = gb_model.predict(X_test)
```

```
In [ ]: accuracy = accuracy_score(y_test, y_pred)
```

```
In [ ]: accuracy
```

```
Out[ ]: 0.9666666666666667
```

```
In [ ]: xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, us
```

```
In [ ]: xgb_model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [08:34:14] WARNING: /workspace/src/learner.cc:740:  
Parameters: { "use_label_encoder" } are not used.  
  
warnings.warn(msg, UserWarning)
```

Out[]:

XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='mlogloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.1, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
```

In []: `y_pred = xgb_model.predict(X_test)`

In []: `accuracy = accuracy_score(y_test, y_pred)`

In []: `accuracy`

Out[]: 0.9333333333333333

In []: